

Zehnora

Setup & Deployment Guide From cloning the repository to a public API

Zehnora Desktop (Mac & Windows) · Zehnora API (Windows GPU PC)

Version Demo 1 · 23 September 2026

Repository: github.com/Inshal-Amir/Zehnora (branch `main`)

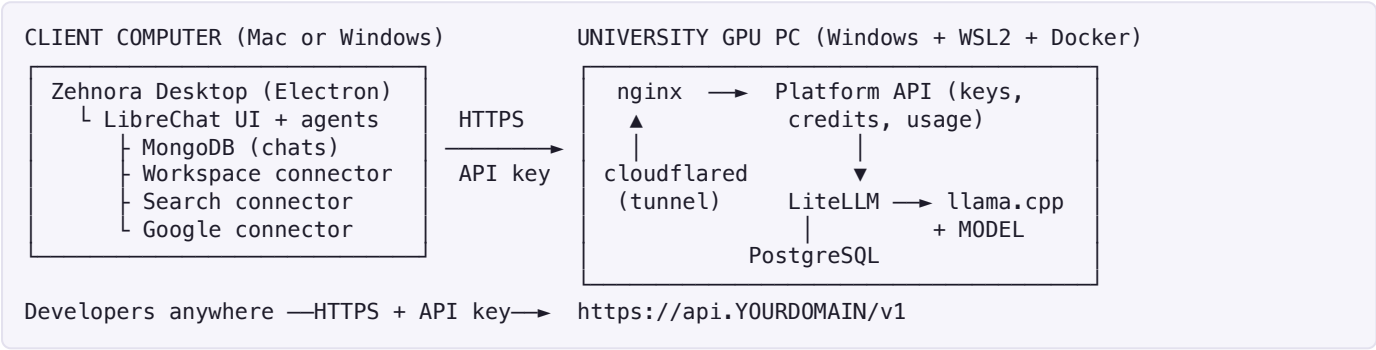
Chat UI based on LibreChat v0.8.7 (MIT)

MAC **WINDOWS** **BOTH** tags show where each step runs

1. What you are building

Zehnora is two connected products that share one repository:

- **Zehnora API** — a developer platform: sign-up, administrator-granted credits, API keys, usage reporting, and an OpenAI-compatible model API. It runs on the university **GPU PC**.
- **Zehnora Desktop** — a branded chat app (LibreChat inside an Electron window) that works on files, runs commands, searches the web and connects to Google Workspace. It runs on each **client computer** (your Mac or a Windows PC).



Golden rules

- The **model is downloaded only on the GPU PC**, never on the Mac.
- The Mac is for development and for the Desktop app; it may use a small stand-in model for testing.
- Secrets (passwords, API keys, tokens) live in files with owner-only permissions and are never committed to Git.

2. What you need before you start

Item	Where	Notes
Private GitHub repository	Both	Holds the code. Model files and databases are never stored in Git.
Git, Node.js 24.16, uv (Python), MongoDB Community	Client computer	Installed in section 3/5.
Windows 10/11 + NVIDIA GPU + WSL2 + Docker Desktop	GPU PC	16 GB VRAM target (RTX 4070 Ti SUPER).
About 60 GB free disk	GPU PC	Model ~10 GB, container images ~15 GB, database + backups.
A domain name + access to its DNS	For public access	Used as <code>api.YOURDOMAIN</code> and <code>console.YOURDOMAIN</code> .
Cloudflare account (free)	For public access	Runs the named tunnel; no router ports are opened.
Google Cloud OAuth client (optional)	Client computer	Only for Gmail/Drive/Calendar/Docs/Sheets features.

3. **MAC** Set up a client computer (Mac)

3.1 Install the basic tools

```
# Xcode command line tools (git, compilers)
xcode-select --install
# uv = Python version and environment manager
curl -LsSf https://astral.sh/uv/install.sh | sh
# MongoDB Community Server (database for chats), via Homebrew
brew tap mongodb/brew && brew install mongodb-community
```

3.2 Clone the repository

```
cd ~/Desktop
git clone https://github.com/Inshal-Amir/Zehnora.git zehnora
cd zehnora
```

3.3 Install Node.js 24.16.0 inside the project

The project uses its own Node so your system Node stays untouched.

```
mkdir -p .local-dev/tools && cd .local-dev/tools
curl -LO https://nodejs.org/dist/v24.16.0/node-v24.16.0-darwin-x64.tar.gz
curl -LO https://nodejs.org/dist/v24.16.0/SHASUMS256.txt
grep node-v24.16.0-darwin-x64.tar.gz SHASUMS256.txt | shasum -a 256 -c # must print OK
tar xzf node-v24.16.0-darwin-x64.tar.gz && rm node-v24.16.0-darwin-x64.tar.gz
export PATH="$PWD/node-v24.16.0-darwin-x64/bin:$PATH" && node -v
cd ~/Desktop/zehnora
```

On an Apple Silicon Mac use `node-v24.16.0-darwin-arm64.tar.gz` instead.

3.4 Build LibreChat (the chat interface)

```
npm ci # installs dependencies (several minutes)
npm run frontend # builds the web interface
```

3.5 Create the Python environments

```
for d in zehnora/platform-api zehnora/infra/litellm zehnora/infra/mock-inference \
        zehnora/connectors/workspace zehnora/connectors/search zehnora/connectors/google
zehnora/tests; do
  (cd $d && uv sync --frozen)
done
# LiteLLM needs its database client generated once (and again after rebuilding its .venv)
(cd zehnora/infra/litellm && PATH="$PWD/.venv/bin:$PATH" \
  PRISMA_NODEENV_CACHE_DIR=../../.local-dev/prisma-nodeenv
PRISMA_BINARY_CACHE_DIR=../../.local-dev/prisma-binaries \
  prisma generate --schema .venv/lib/python3.12/site-packages/litellm/proxy/schema.prisma)
# browser used for page checks and tests
PLAYWRIGHT_BROWSERS_PATH=$PWD/.local-dev/playwright-browsers \
  uv run --project zehnora/tests playwright install chromium
```

3.6 Install the Electron shell

```
cd zehnora/desktop && npm install && node node_modules/electron/install.js && cd ../../
```

4. MAC Optional: run the platform locally for development

You only need this if you want the whole API platform on the Mac (useful before the GPU PC is ready). If your GPU PC is already serving, skip to section 5 and point the Desktop app at it.

4.1 Local database and services

The development stack uses a portable PostgreSQL inside the project folder, then starts everything:

```
zehnora/scripts/mac/dev-platform.sh start mock          # fake model: instant, replies labelled  
MOCK  
zehnora/scripts/mac/dev-platform.sh start dev-local-4b  # small real model on the CPU (slow)  
zehnora/scripts/mac/dev-platform.sh status  
zehnora/scripts/mac/dev-platform.sh stop
```

What starts: PostgreSQL (5433), the model upstream, LiteLLM gateway (4000), the platform API (8200) and the portal (5173). Everything listens on 127.0.0.1, meaning this Mac only.

4.2 Create the first administrator and the model entry

```
cd zehnora/platform-api  
uv run python -m app.cli bootstrap-admin --email admin@example.com  # asks for a password  
uv run python -m app.cli create-gateway-key                        # prints a key  
# paste that key into zehnora/platform-api/.env as ZEHNORA_PLAYGROUND_GATEWAY_KEY, then restart  
the API
```

Open <http://127.0.0.1:5173>, log in as the admin, and you can create customers, grant credits and issue API keys.

5. BOTH Run Zehnora Desktop

5.1 Give the app an API key

In the portal (your own server, or the GPU PC once deployed): **API keys** → **Create key**. Copy the key; it is shown once.

5.2 Start it MAC

```
# store the key for the app (owner-only file)  
mkdir -p .local-dev/client/secrets && chmod 700 .local-dev/client/secrets  
printf '%s' 'sk-YOUR-KEY' > .local-dev/client/secrets/zehnora-api-key  
chmod 600 .local-dev/client/secrets/zehnora-api-key  
  
# point it at your server (leave out to use a local platform)  
export ZEHNORA_API_BASE=https://api.YOURDOMAIN/v1  
zehnora/scripts/mac/start-desktop.sh
```

5.3 Start it WINDOWS

```
powershell -ExecutionPolicy Bypass -File zehnora\scripts\windows\Start-Desktop.ps1
```

Prerequisites on Windows: Git, Node 24.16.0, uv, MongoDB Community (`mongod.exe` on PATH), the same `npm ci / npm run frontend / uv sync` steps as sections 3.4–3.6. Put the API key in `%APPDATA%\Zehnora\secrets\zehnora-api-key`, or paste it in the app's first-run window.

5.4 Create the local chat account (first run only)

The Desktop chat keeps its own local account, separate from the portal account.

```
# Mac (from the repository root, with the services running)
MONGO_URI=mongodb://127.0.0.1:27028/ZehnoraDesktop \
CONFIG_PATH=$PWD/.local-dev/client/librechat.yaml \
.local-dev/tools/node-v24.16.0-darwin-x64/bin/node config/create-user.js you@example.com "Your
Name" you --email-verified=true
```

5.5 First use

1. The top bar shows your **workspace folder** and **Model API: online**. Use **Change folder** to pick which folder the assistant may touch.
2. Log in with the local account, pick the **Zehnora Coder** agent, and ask for something, for example: *“Create a folder demo-notes with a README.md explaining Zehnora, then list it.”*
3. Normal build and test commands (npm, python, pytest...) run automatically. Anything else shows an **approval window** with the exact command and Deny / Approve once.

Safety: the assistant can only touch the selected workspace folder. Approved commands run with your user permissions, so read them before approving. This is not a full sandbox.

6. WINDOWS Deploy the server on the GPU PC

This is where the real model lives. Every block says whether it is **PowerShell** (Windows) or **WSL** (the Ubuntu terminal).

6.1 Check the machine — PowerShell

```
powershell -ExecutionPolicy Bypass -File zehnora\scripts\windows\Preflight.ps1 -GpuContainerTest
```

It checks Windows, RAM, disk, the NVIDIA driver, WSL2, Docker and port 8080, and writes `preflight-result.json`. Fix anything marked CHECK:

- **NVIDIA driver:** install the latest Windows driver. Never install Linux display drivers inside WSL — WSL uses the Windows driver.
- **WSL2:** `wsl --install -d Ubuntu-24.04` in an admin PowerShell, then reboot.
- **Docker Desktop:** install, then enable *Use the WSL 2 based engine* and WSL integration for Ubuntu.

6.2 Get the code — WSL

```
sudo usermod -aG docker $USER          # let your user run docker; then close and reopen
Ubuntu
docker run --rm --gpus all ubuntu nvidia-smi # must show the RTX card from inside a container
sudo apt-get update && sudo apt-get install -y git curl openssl python3
curl -LsSf https://astral.sh/uv/install.sh | sh && source $HOME/.local/bin/env
cd ~ && git clone https://github.com/Inshal-Amir/Zehnora.git zehnora
cd ~/zehnora
```

Keep it in `~/zehnora` (the Linux filesystem), not under `/mnt/c`, for speed.

6.3 Create the server secrets — WSL

```
zehnora/scripts/server/init-secrets.sh
mkdir -p ~/zehnora-models
sed -i "s#^ZEHNORA_MODEL_DIR=.*#ZEHNORA_MODEL_DIR=$HOME/zehnora-models#" .server-
secrets/server.env
```

`OWNER_DOMAIN` in `server.env` is set later, in section 7, once the domain is known.

6.4 The model and engine

The default is **Qwen3.6-35B-A3B** (a mixture-of-experts coding model: 35B parameters, 3B active per token; SWE-bench Verified 73.4%) served by **llama.cpp**. The 22.4 GB 4-bit file does not fit in 16 GB of VRAM, so llama.cpp keeps the attention layers on the GPU and puts the remaining experts in system RAM (`--fit on`). Thinking mode is on, with the model card's coding sampling (temperature 0.6, top-p 0.95). Clients can switch thinking off per request with `"chat_template_kwargs": {"enable_thinking": false}`.

For many simultaneous users, set `ZEHNORA_ENGINE=vllm` with a model that fits in VRAM (see the comments in `server.env`) and verify the tool-call parser with `docker run --rm vllm/vllm-openai:v0.29.0 --help | grep -A6 -- '--tool-call-parser'`.

6.5 Download the model — WSL

```
zehnora/scripts/server/download-model.sh
```

Downloads the pinned file (about 22.4 GB, resumable if interrupted) to your SSD outside Git, checks its SHA-256 against the value in `server.env`, and writes a provenance file. Needs about 30 GB free.

6.6 Build the portal and start everything — WSL

```
# Node 24 for building the portal (inside WSL, once)
cd ~ && curl -LO https://nodejs.org/dist/v24.16.0/node-v24.16.0-linux-x64.tar.xz
curl -LO https://nodejs.org/dist/v24.16.0/SHASUMS256.txt
grep node-v24.16.0-linux-x64.tar.xz SHASUMS256.txt | sha256sum -c - # must print OK
mkdir -p ~/.local/node && tar xJf node-v24.16.0-linux-x64.tar.xz -C ~/.local/node --strip-components=1
echo 'export PATH="$HOME/.local/node/bin:$PATH"' >> ~/.bashrc && export
PATH="$HOME/.local/node/bin:$PATH"
rm node-v24.16.0-linux-x64.tar.xz SHASUMS256.txt

cd ~/zehnora/zehnora/portal && npm ci && npm run build && cd ~/zehnora
zehnora/scripts/server/start.sh # local only; no public access yet
zehnora/scripts/server/health.sh
```

`start.sh` builds the images, starts PostgreSQL and the model (loading 22 GB takes a few minutes), waits for each service to become healthy, then LiteLLM, the platform API and nginx on `127.0.0.1:8080`. It prints which model is deployed and the GPU memory in use.

6.7 First administrator, model catalog, playground key — WSL

```
zehnora/scripts/server/bootstrap-admin.sh admin@YOURDOMAIN

docker exec zehnora-platform-api-1 python -m app.cli seed-model --alias zehnora-coder \
  --identity "Qwen3.6-35B-A3B UD-Q4_K_XL GGUF (unsloth @ a483e9e), llama.cpp v0.4.1, RTX 4070 Ti SUPER + RAM" \
  --context 65536 --max-output 16384 --default-output 8192

docker exec zehnora-platform-api-1 python -m app.cli create-gateway-key
nano .server-secrets/platform.env # paste as ZEHNORA_PLAYGROUND_GATEWAY_KEY, set
ZEHNORA_PUBLIC_API_BASE
zehnora/scripts/server/start.sh # applies the new platform.env
```

Output limits are large because thinking tokens count as output: a small `max_tokens` can be used up by thinking before any answer appears.

6.8 Test locally before going public — WSL

```
curl -s -H "Host: api.YOURDOMAIN" http://127.0.0.1:8080/v1/models # expect 401 without a key
```

Then temporarily add `127.0.0.1 console.YOURDOMAIN api.YOURDOMAIN` to

`C:\Windows\System32\drivers\etc\hosts`, open the console in a browser, register a customer, grant credits, create a key, and run:

```
ZEHNORA_BASE_URL=http://api.YOURDOMAIN:8080/v1 ZEHNORA_API_KEY=sk-... \
uv run --project zehnora/tests python zehnora/scripts/test-public-api.py --no-thinking
```

`--no-thinking` is needed because the test uses short answer budgets. Remove the hosts lines afterwards.

7. Put it on the internet (Cloudflare Tunnel)

A tunnel makes an **outgoing** connection from the PC to Cloudflare, so you do not open any port on the university router or firewall.

1. Add your domain to Cloudflare (change the nameservers at your registrar). Do not delete existing DNS records you rely on.
2. Cloudflare dashboard → **Zero Trust** → **Networks** → **Tunnels** → **Create a tunnel** (type: cloudflared). Copy the tunnel **token**.
3. **WSL**: paste the token into `.server-secrets/cloudflared_token`.
4. In the tunnel's **Public hostnames**, add two entries, both pointing to `http://nginx:8080`:
 - `api.YOURDOMAIN` — the model API for developers
 - `console.YOURDOMAIN` — the web portal
5. **WSL**: `zehnora/scripts/server/start.sh --with-tunnel`

Do not put Cloudflare Access (the browser login page) in front of `api.` — SDK clients cannot log in through a web page, and every request would fail.

7.1 Test from a different internet connection

```
export ZEHNORA_BASE_URL=https://api.YOURDOMAIN/v1
export ZEHNORA_API_KEY=sk-...
uv run --project zehnora/tests python zehnora/scripts/test-public-api.py --no-thinking
# revoke the key in the portal, then confirm it stops working:
uv run --project zehnora/tests python zehnora/scripts/test-public-api.py --expect-revoked
```

Also confirm these return “not found”: `https://api.YOURDOMAIN/key/generate` and `https://console.YOURDOMAIN/v1/models`.

7.2 What a developer now uses

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="zehnora-coder",
                  base_url="https://api.YOURDOMAIN/v1",
                  api_key="sk-...", use_responses_api=False)
print(llm.invoke("Write a Python function that validates a task title.").content)
```

The same key works with the OpenAI SDK and plain curl. The base URL always ends at `/v1` — never add `/chat/completions` to it.

7.3 Testing without a domain (temporary)

To practise on a Mac or before the domain is ready, a quick tunnel gives a random temporary address:

```
# terminal 1: publish ONLY the model API (blocks the portal and admin routes)
uv run --project zehnora/platform-api python zehnora/infra/edge-proxy.py --host 127.0.0.1
# terminal 2: temporary public HTTPS address
cloudflared tunnel --url http://127.0.0.1:8300
```

Use the printed `https://...trycloudflare.com/v1` as the base URL. Press Ctrl+C to take it offline. This is for learning only: the address changes each time and your laptop is doing the serving.

8. Daily operations WINDOWS GPU PC

Task	Command (WSL)
Start everything	<code>zehnora/scripts/server/start.sh --with-tunnel</code>
Check health	<code>zehnora/scripts/server/health.sh</code>
Stop (keeps all data)	<code>zehnora/scripts/server/stop.sh</code>
Backup databases	<code>zehnora/scripts/server/backup.sh</code> then encrypt the folder
Update to the latest version	<code>git pull (branch main)</code> then <code>start.sh</code>
View logs	<code>docker compose -f zehnora/infra/server/compose.yaml logs -f platform-api</code>

Never run `docker compose down -v` — the `-v` deletes the database volume with all accounts, credits and usage history.

Keep it available: set Windows sleep to “Never” while hosting, and remember that after a reboot someone must log in so Docker Desktop starts, then run `start.sh --with-tunnel`. This is not an unattended server.

9. Troubleshooting

Symptom	Cause and fix
Portal login fails	The portal and the Desktop app have separate accounts. Use the portal account. After 5 wrong passwords an email is blocked for 15 minutes.
402 insufficient credits	The account has no credits. An administrator grants them in Users & credits .
401 invalid API key	Key revoked, expired, or copied incompletely. Create a new one.
503 billing database unavailable	PostgreSQL is down. The system refuses work rather than running untracked requests. Check <code>health.sh</code> .
429	Too many requests at once for this demo capacity. Retry shortly.
Model will not start / out of memory	See <code>docker logs zehnoramodel-1</code> . Lower <code>ZEHNORA_MAX_MODEL_LEN</code> (e.g. 32768) in <code>server.env</code> , then <code>start.sh</code> . If system RAM runs out, give WSL more memory in <code>%UserProfile%\wslconfig</code> (<code>[wsl2] memory=48GB</code>), run <code>wsl --shutdown</code> , restart Docker Desktop.
Answer is empty, <code>finish_reason</code> is <code>length</code>	Thinking used the whole <code>max_tokens</code> budget. Raise <code>max_tokens</code> , or send <code>"chat_template_kwargs": {"enable_thinking": false}</code> .
Model gives text instead of using a tool	vLLM engine: wrong <code>ZEHNORA_TOOL_PARSER</code> (section 6.4). llama.cpp: check the model file and <code>docker logs zehnoramodel-1</code> .
Tunnel shows “down” in Cloudflare	University network may block it. Report it; do not attempt to bypass network controls.
Desktop: “Model API: unreachable”	Wrong <code>ZEHNORA_API_BASE</code> , server stopped, or no internet. Test with <code>curl \$ZEHNORA_API_BASE/models</code> .
Assistant says a command needs approval but no window appears	The Desktop app is not running (the approval window lives in it). Start it, or use an ordinary <code>build/test</code> command.
Mac: <code>litellm</code> did not become ready, log says <i>Unable to find Prisma binaries</i>	The LiteLLM environment was rebuilt without its database client. Run the <code>prisma generate</code> step from section 3.5, then start again.
Port already in use	The scripts refuse to touch processes they do not own. Stop the other program, or change the port at the top of <code>dev-platform.sh</code> / <code>start-desktop.sh</code> .

10. Reference

Ports (all local unless tunnelled)

Port	Service	Where
5173	Portal (development)	Client / dev Mac
8200	Platform API	Dev Mac / GPU PC (internal)
8300	Edge proxy (public API only)	Mac, for tunnelling practice
4000	LiteLLM gateway	Internal only
8000	Model server (llama.cpp or vLLM)	GPU PC, internal only
8080	nginx (behind the tunnel)	GPU PC
3090	LibreChat (Desktop)	Client computer
27028	MongoDB (chats)	Client computer
8888	SearXNG (web search)	Client computer
3099	Approval bridge	Client computer

Important files

Path	Contents
<code>.server-secrets/</code> GPU PC	Database passwords, gateway key, tunnel token, <code>server.env</code> . Never committed.
<code>.local-dev/</code> MAC	Development database, tools, logs and secrets. Never committed.
<code>~/Desktop/Zehnora-Workspace</code> CLIENT	Default folder the Desktop assistant works in (change it in the app).
<code>~/ .zehnora/</code> CLIENT	Chosen workspace folder and connector logs.
<code>zehnora/docs/</code>	STATUS, ARCHITECTURE, API-CONTRACT, CREDITS-AND-RECOVERY, GOOGLE-SETUP, MODEL-EVALUATION, KNOWN-LIMITATIONS, DEMO-SCRIPT.
<code>zehnora/scripts/</code>	<code>mac/</code> , <code>windows/</code> and <code>server/</code> start, stop, health and deployment scripts.

Security summary

- Only two public routes exist: `GET /v1/models` and `POST /v1/chat/completions`. The gateway, model, databases and desktop tools are never routed publicly.
- Credits are reserved before generation and settled on actual usage; the ledger cannot be edited or deleted.
- API keys are stored as one-way fingerprints — a lost key cannot be recovered, only replaced.
- The Desktop assistant is limited to the chosen workspace folder; other commands need your approval.
- Web pages fetched by the assistant are treated as untrusted data, and private network addresses are blocked.

Read `zehnora/docs/KNOWN-LIMITATIONS.md` for what is not yet implemented or tested, including Windows testing, the Google connector and GPU acceptance tests.